



HDFS

Парадигма MapReduce

Лектор: Воронов Ф.А.
2026 год



Локальный диск

- Быстрая работа с данными (лежат локально)
- Произвольный доступ
- Любые типы и размеры файлов
- Любые изменения данных

Локальный диск

- Быстрая работа с данными (лежат локально)
- Произвольный доступ
- Любые типы и размеры файлов
- Любые изменения данных
- Ограничение размеров
- Проблемы с НА

OLTP

- Быстрая работа на коротких запросах
- Произвольный доступ к строкам
- Хранят таблицы и индексы
- Транзакционные изменения данных

OLTP

- Быстрая работа на коротких запросах
- Произвольный доступ к строкам
- Хранят таблицы и индексы
- Транзакционные изменения данных

- Не адаптировано под большие сканы
- Построчный, а не поколоночный формат

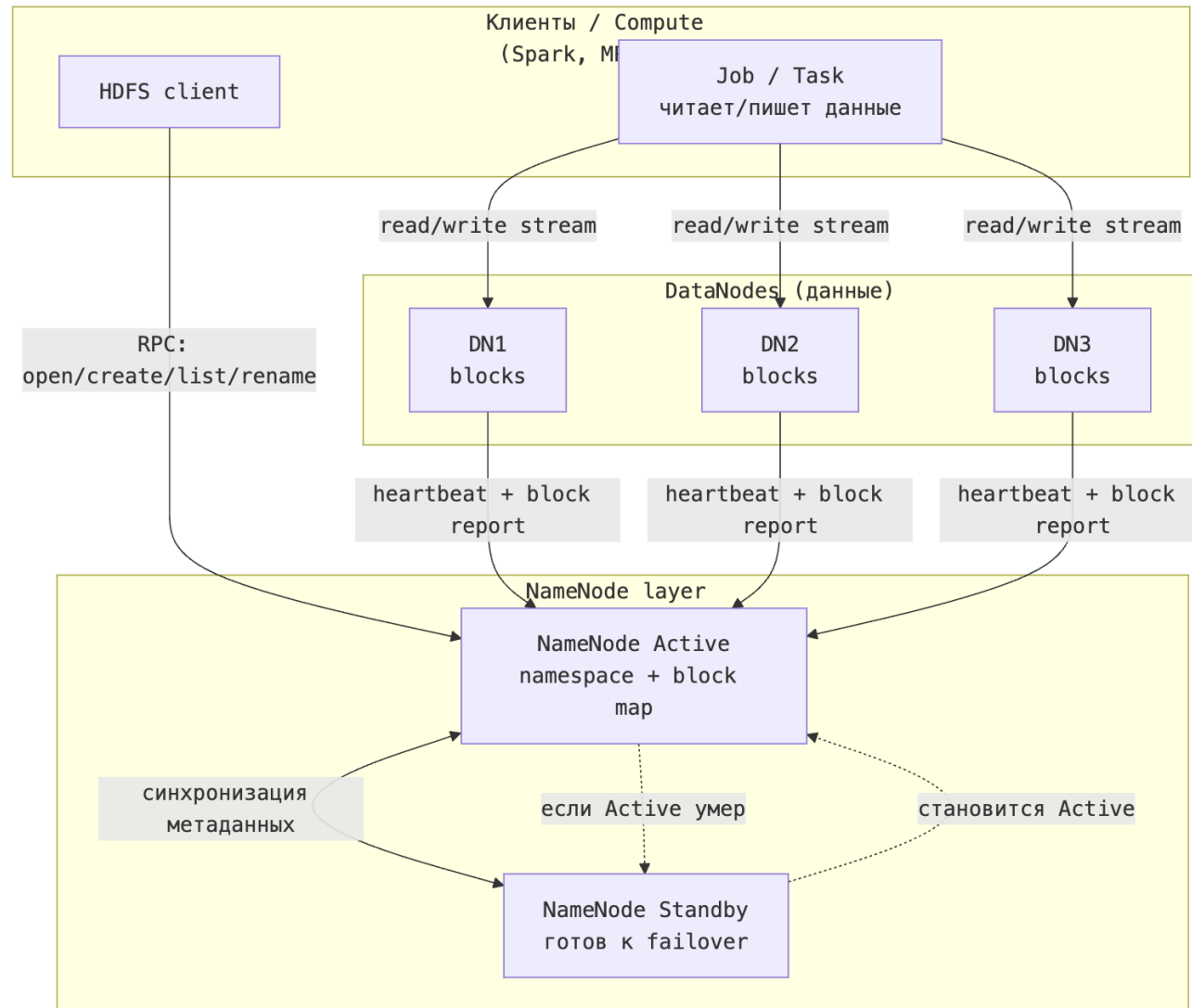
HDFS

- Большая пропускная способность на больших данных
- Адаптировано под длинные последовательные чтения
- Большие файлы
- Неизменяемые файлы (или только append)
- Ограничения на скачивание больших объёмов данных вне кластера

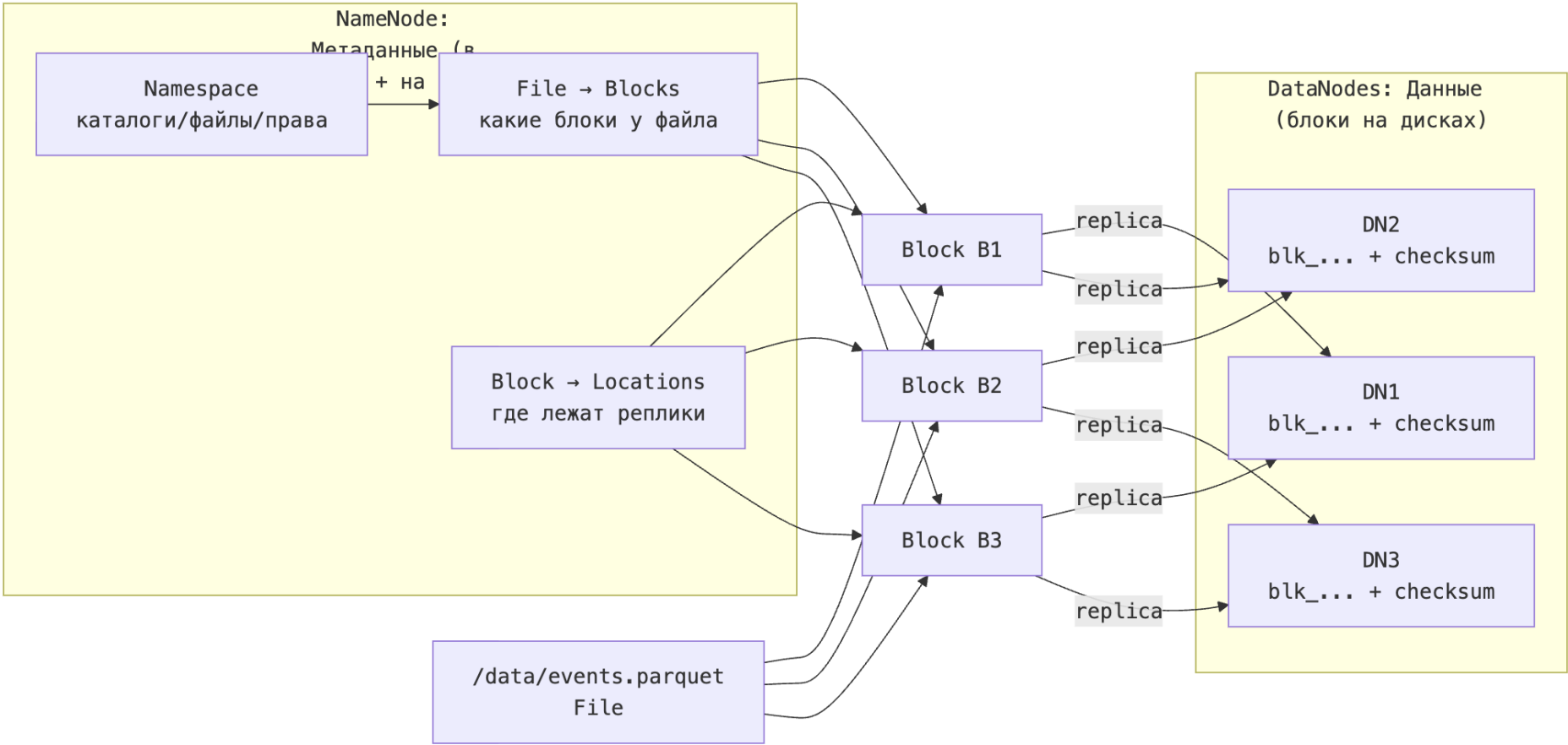
HDFS

- `hdfs dfs -ls <path>` — список файлов/директорий
- `hdfs dfs -mkdir <path>` — создать директорию
- `hdfs dfs -put <local> <hdfs>` — загрузить (алиас: `-copyFromLocal`)
- `hdfs dfs -get <hdfs> <local>` — скачать (алиас: `-copyToLocal`)
- `hdfs dfs -cat <path>` — вывести файл
- `hdfs dfs -head <path>` — первые ~1KB
- `hdfs dfs -cp <src> <dst>` — копия
- `hdfs dfs -mv <src> <dst>` — перемещение/переименование

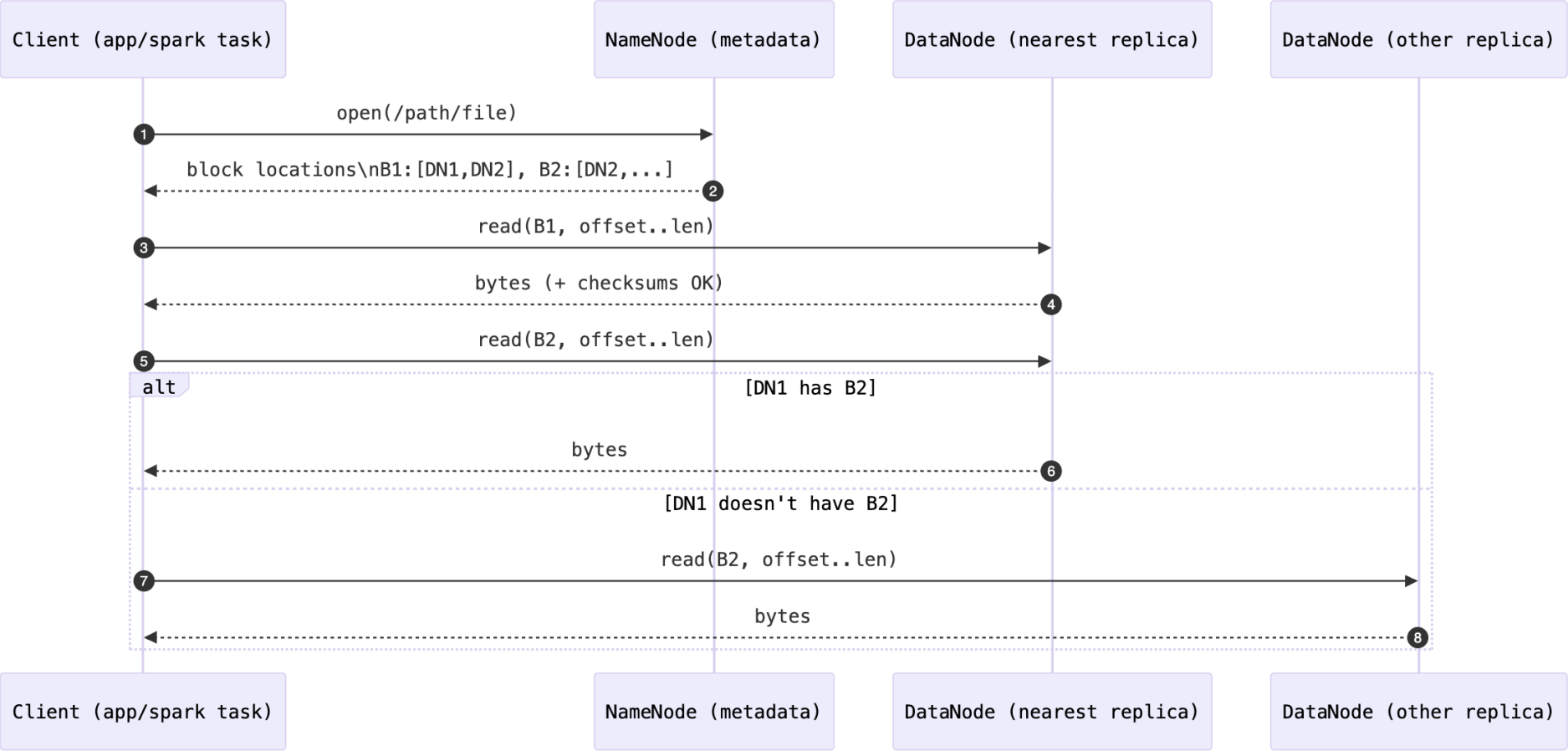
Устройство HDFS



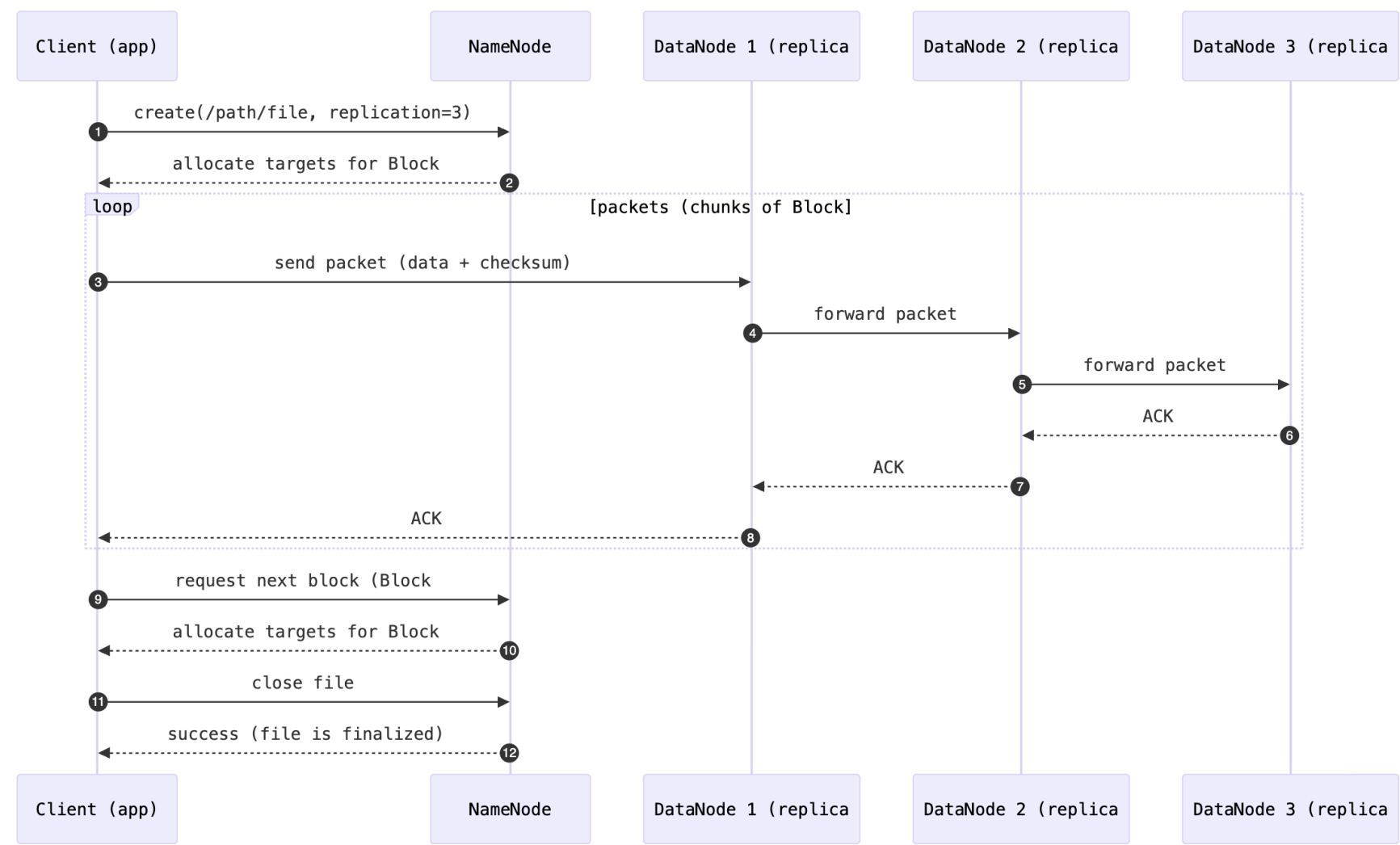
Блочная система



Чтение с клиента



Запись в HDFS



Особенности HDFS

Small files problem (много маленьких файлов)

- Каждый файл/директория = метаданные (inode, список блоков, права) в памяти **NameNode**.
- Миллионы мелких файлов → растёт RAM/GC/нагрузка на NN, операции `ls`, `rename`, `rm` замедляются.
- Часто хуже всего: `_много файлов по 1–10 МБ_` — и место вроде есть, а NN “умирает”.

Latency (не про миллисекунды)

- HDFS оптимизирован под **пропускную способность**, а не под быстрый ответ на короткий запрос.
- Открытие файла/поиск блоков/сетевые хопы → “дорого” для маленьких операций.

Поэтому HDFS плох для OLTP/интерактива “дай одну строку за 5 мс”.

Особенности HDFS

Random writes / обновления “в середине файла”

- Нельзя “перезаписать байты по смещению” как на обычной FS.
- Нормальный сценарий: write once + append (или immutable файлы).
- “Обновления” обычно делаются как: _записали новый файл/партицию → переключили ссылку/метаданные → старое удалили позже_.

3× storage overhead (репликация)

- Replication factor 3 → примерно 3× место и больше трафика на запись.
- Это плата за простоту и надёжность на дешёвом железе.
- Есть альтернативы (erasure coding), но они сложнее по CPU/latency и не всегда подходят.

Parquet

- Колоночный формат (column-oriented)
- Encoding + Compression
- Статистики/индексы для быстрого пропуска

Logical table representation

a	b	c
a1	b1	c1
a2	b2	c2
a3	b3	c3
a4	b4	c4
a5	b5	c5

Row Layout



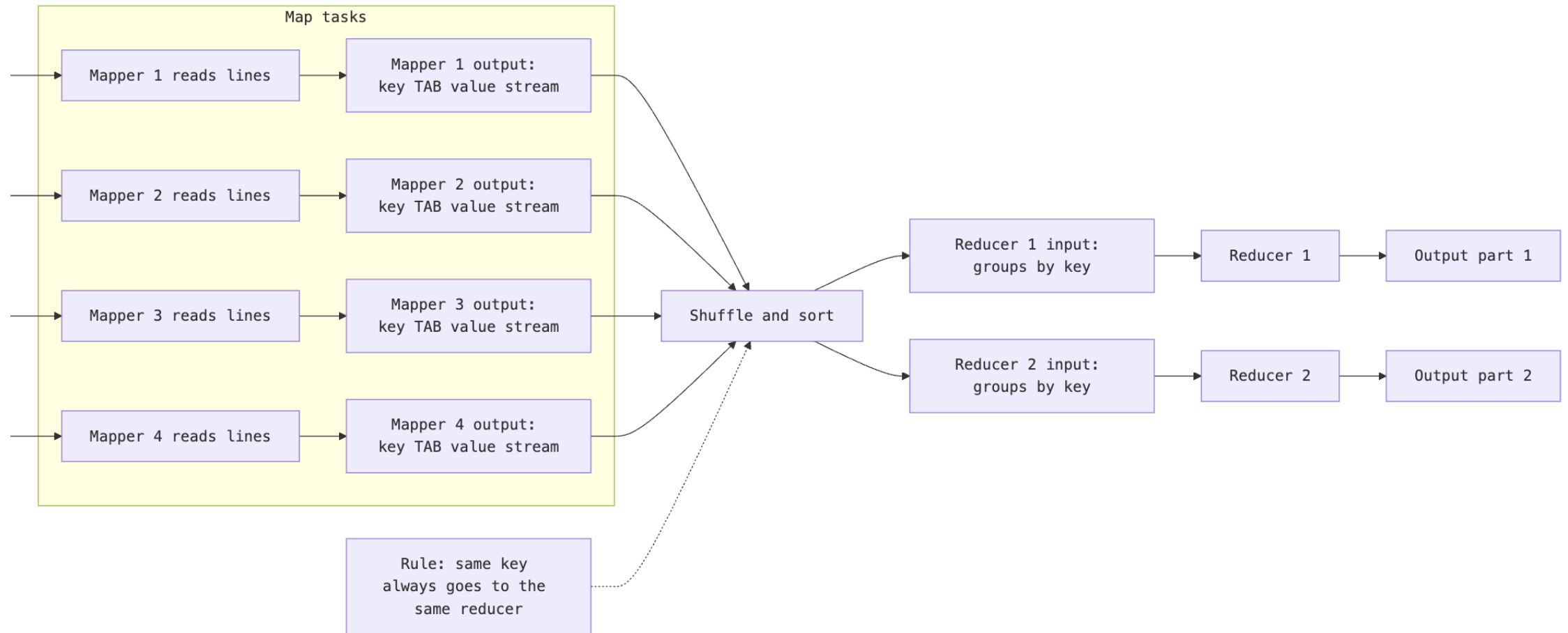
Column Layout



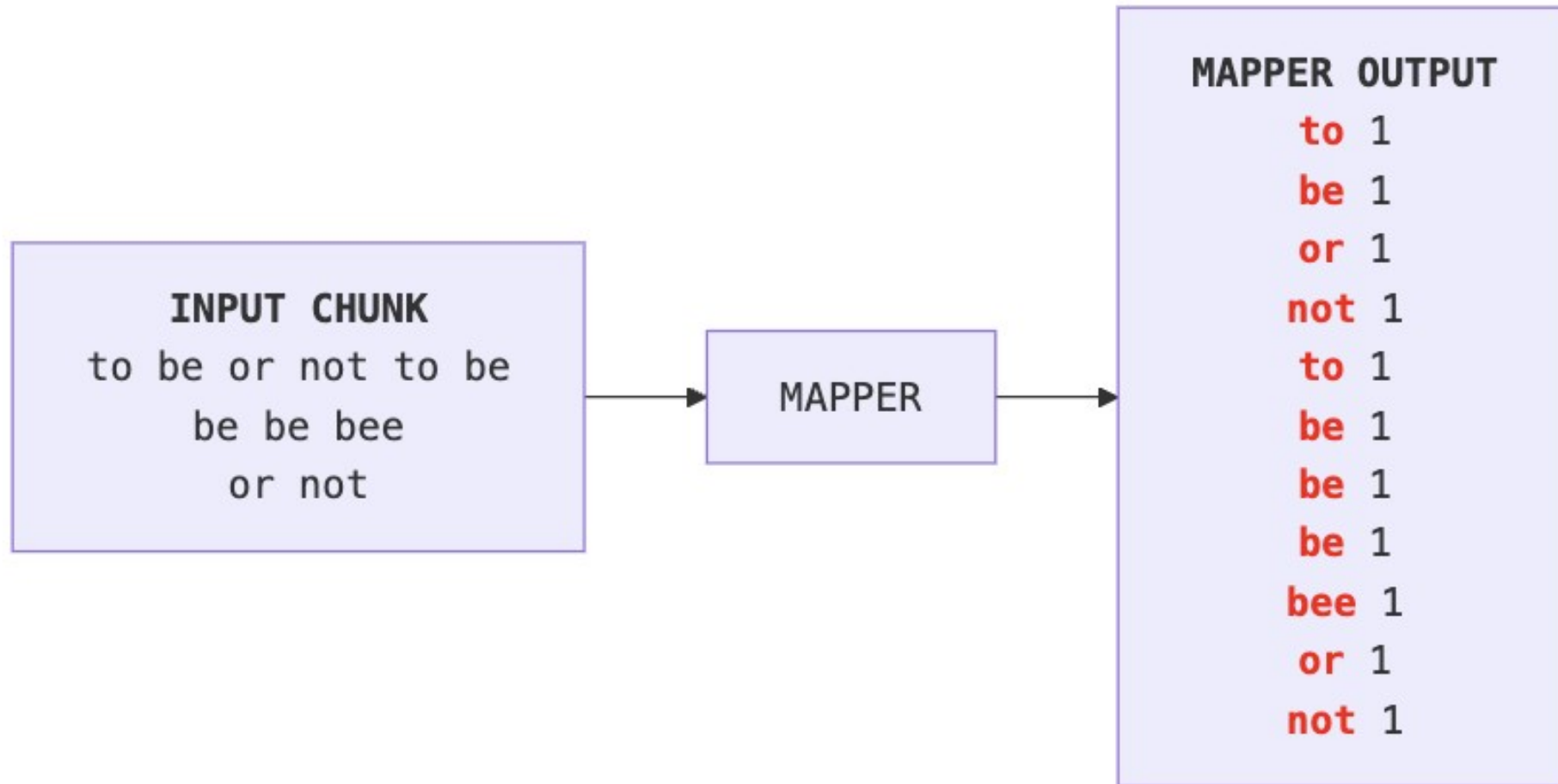
encoding



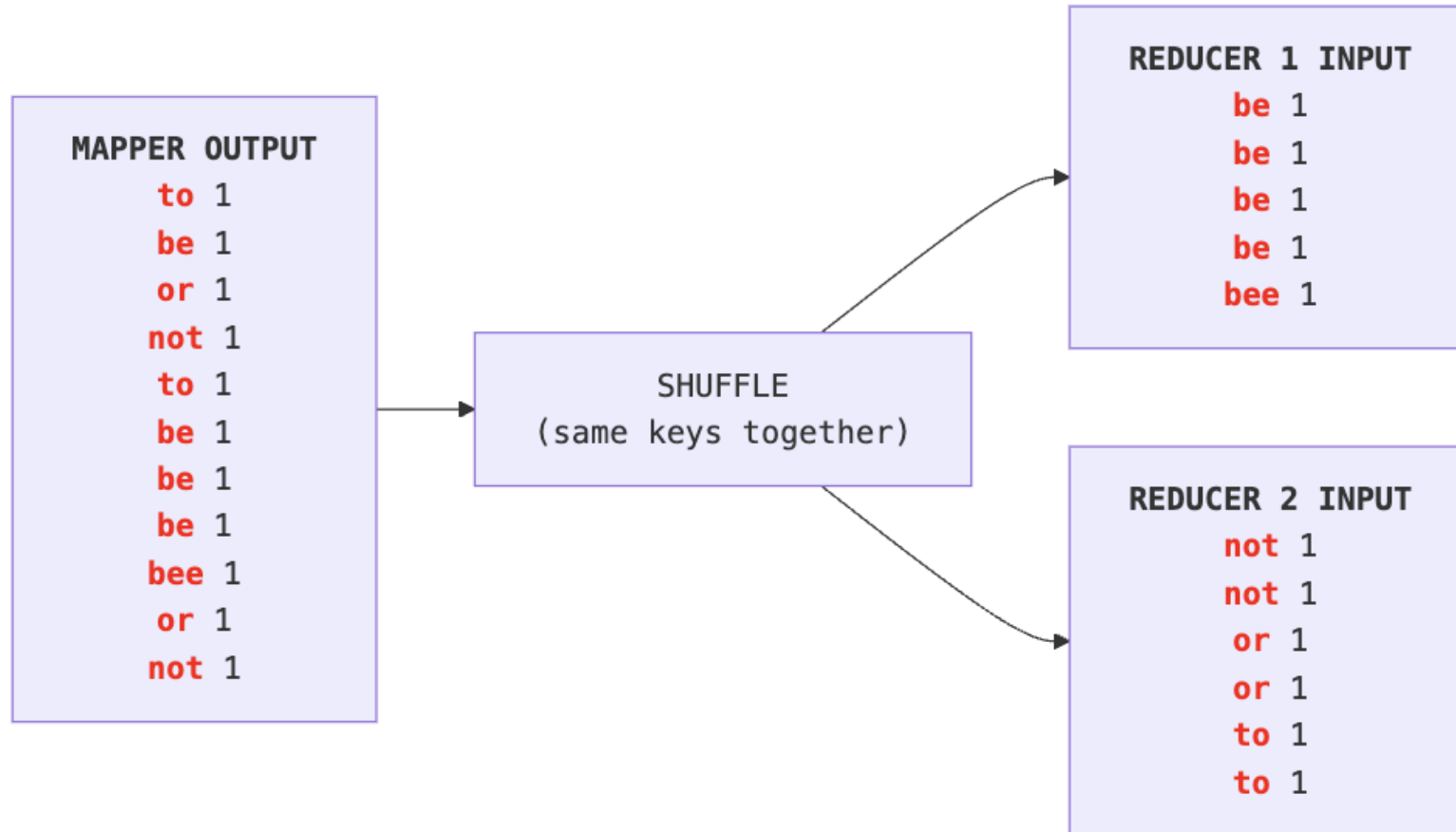
Map Reduce



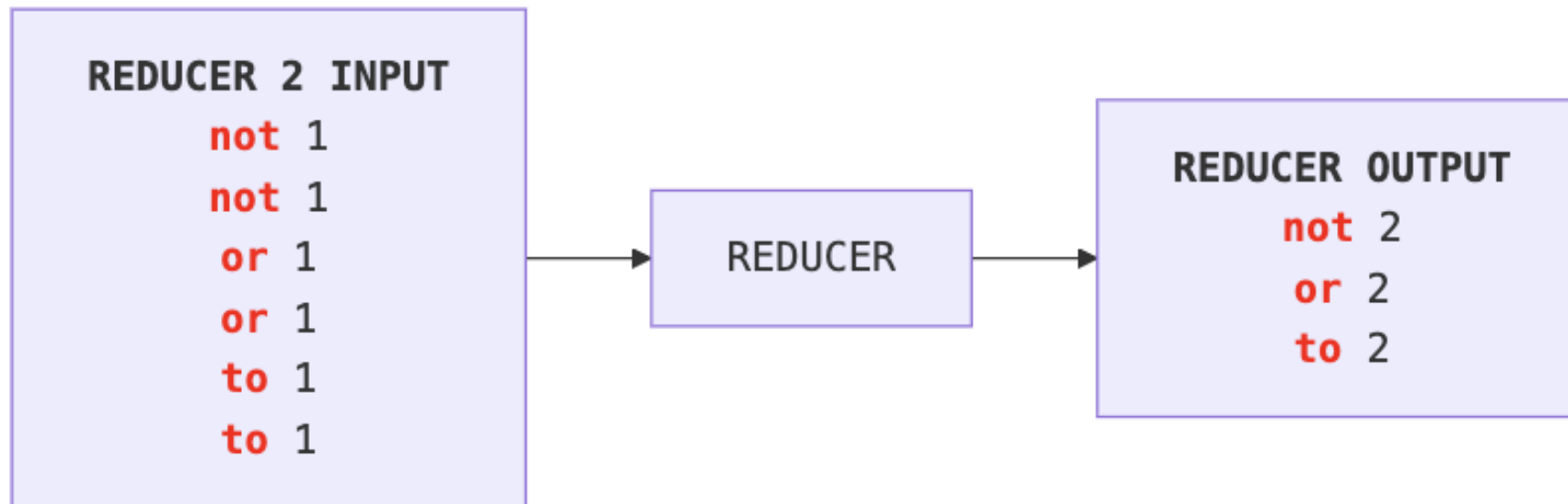
WordCount / Mapper



WordCount / Shuffle



WordCount / Reducer



Нadoop: локальная установка

• •

```
# скачать хадуп
curl -L -o hadoop.tar.gz "https://archive.apache.org/dist/hadoop/common/hadoop-3.4.2/hadoop-3.4.2.tar.gz"

# распаковать
tar -xzf hadoop.tar.gz

# прописать переменные окружения
export JAVA_11_HOME=$(/usr/libexec/java_home -v 11)

export JAVA_HOME=$JAVA_11_HOME
export PATH="$JAVA_HOME/bin:$PATH"

export HADOOP_HOME=~/.Workspace/tmp/pyhadoop/hadoop-3.4.2
export PATH="$HADOOP_HOME/bin:$HADOOP_HOME/sbin:$PATH"

# прописать JAVA_HOME в hadoop-3.4.2/etc/hadoop/hadoop-env.sh
```



Спасибо за внимание!

